

PRINCIPIOS SOLID

- **PRINCIPIO DE RESPONSABILIDAD ÚNICA (SRP)**

Una clase debe de tener una única función y, por lo tanto, debe de tener una sola razón para ser modificada.

[customer-service]

- En el package `service` se crearon dos interfaces: `CustomerService` y `AccountValidationService`. La primera define un contrato para listar, registrar y eliminar clientes; mientras que la segunda para la validación de cuentas.
- El `GlobalExceptionHandler` se encarga exclusivamente del manejo centralizado de excepciones contiene `handleMethodArgumentNotValidException`, `handleDniAlreadyExistsException`, `handleCustomerNotFoundException` y `handleCustomerHasActiveAccountsException`.

[account-service]

- En el package `service` se crearon dos interfaces: `AccountService` y `TransactionService`. La primera maneja las funcionalidades de registrar, listar y eliminar cuentas; mientras que la segunda presenta las funcionalidades de depositar y retirar el saldo de las cuentas.
- En el package útil se creó dos clases `AccountConstants` y `AccountNumberGenerator`, cada una con responsabilidades específicas
- Las clases de excepción manejan errores específicos de forma aislada.
- El `GlobalExceptionHandler` se encarga exclusivamente del manejo centralizado de excepciones contiene `handleMethodArgumentNotValidException`, `handleAccountNotFoundException`, `handleInsufficientBalanceException` y `handleCustomerNotFoundException`.

[transaction-service]

- En el package `service` se crearon dos interfaces: `TransactionService`, contiene la lógica de negocio para procesar transacciones.
- `AccountWebClient`, proporciona un cliente HTTP reactivo para comunicarse con el microservicio de cuentas, permitiendo realizar operaciones sobre cuentas bancarias desde el servicio de transacciones
- Las clases de excepción manejan errores específicos de forma aislada.
- El `GlobalExceptionHandler` se encarga exclusivamente del manejo centralizado de excepciones contiene `handleValidationException`, `handleAccountNotFound`, `handleInsufficientBalance` y `handleTransactionException`.

- **PRINCIPIO DE ABIERTO/CERRADO (OCP)**

Las entidades de software deben estar abiertas para extensión pero cerradas para modificación.

[customer-service]

- El uso de grupos de validación (`CreateCustomerValidationGroup`) permite extender las reglas de validación sin modificar las existentes.
- La implementación del patrón Strategy permite agregar nuevas validaciones sin modificar el código existente.
- El uso de DTOs permite extender los modelos de datos sin afectar a las entidades principales.

[account-service]

- Las interfaces `AccountService` y `TransactionService` definen contratos que pueden ser implementados de diferentes maneras sin modificar el código existente.
- El uso de DTOs (`AccountRequestDTO`, `AccountResponseDTO` y `TransactionRequestDTO`) permiten extender los modelos de datos sin afectar las entidades principales.
- El `AccountMapper` facilita la transformación entre entidades y DTOs, permitiendo añadir nuevas conversiones sin modificar las existentes.

[transaction-service]

- Diseñado para extensión mediante los diferentes tipos de transacciones (depósito, retiro y transferencia), con estrategias de procesamiento extensibles.

● PRINCIPIO DE SUSTITUCIÓN DE LISKOV (LSP)

Los objetos de una superclase deben poder ser reemplazados por objetos de una subclase sin afectar la corrección del programa.

[customer-service]

- `CustomerServiceImpl` y `AccountValidationServiceImpl` pueden ser sustituidas por cualquier otra implementación que cumpla con las interfaces `CustomerService` y `AccountValidationService` respectivamente.

[account-service]

- Las implementaciones `AccountServiceImpl` y `TransactionServiceImpl` pueden ser sustituidas por cualquier otra implementación que cumpla con las interfaces `AccountService` y `TransactionService` respectivamente.
- El uso de interfaces garantiza que cualquier implementación concreta cumplirá con el contrato definido, permitiendo la sustitución transparente.

[transaction-service]

- Las diferentes implementaciones de transacciones son sustituibles entre sí
- Cumplen con el mismo contrato de interfaz.

● PRINCIPIO DE SEGREGACIÓN DE INTERFACES (ISP)

Muchas interfaces específicas son mejores que una interfaz de propósito general.

[customer-service]

- Se han creado interfaces separadas para diferentes responsabilidades: `CustomerService` para gestión de clientes, `AccountValidationService` para validación de cuentas y `ValidationStrategy` para estrategias de validación específicas

[account-service]

- Se han creado interfaces separadas para diferentes responsabilidades: `AccountService` para gestión de cuentas y `TransactionService` para operaciones transaccionales.
- Esto evita que las clases dependan de métodos que no utilizan, permitiendo al controlador implementar solo lo que necesitan.

[transaction-service]

- Presenta interfaces específicas para cada responsabilidad: `TransactionService` para la lógica de negocio, `TransactionRepository` para operaciones de bases de datos y `TransactionStrategy` para estrategias de transacciones específicas.

- **PRINCIPIO DE INVERSIÓN DE DEPENDENCIAS (DIP)**

Debe depender de abstracciones, no de implementaciones concretas.

[customer-service]

- Todas las clases dependen de interfaces abstractas en lugar de implementaciones concretas: como `CustomerController` depende de `CustomerService` (interfaz)

[account-service]

- El `AccountController` depende de las interfaces `AccountService` y `TransactionService` en lugar de sus implementaciones concretas.
- La inyección de dependencias a través de constructores permite cambiar fácilmente las implementaciones.
- La configuración en `RestTemplateConfig` sigue este principio al definir `RestTemplate` como un bean que pueda ser inyectado donde se necesite.

[transaction-service]

- El servicio depende de abstracciones (interfaces) no de implementaciones concretas.

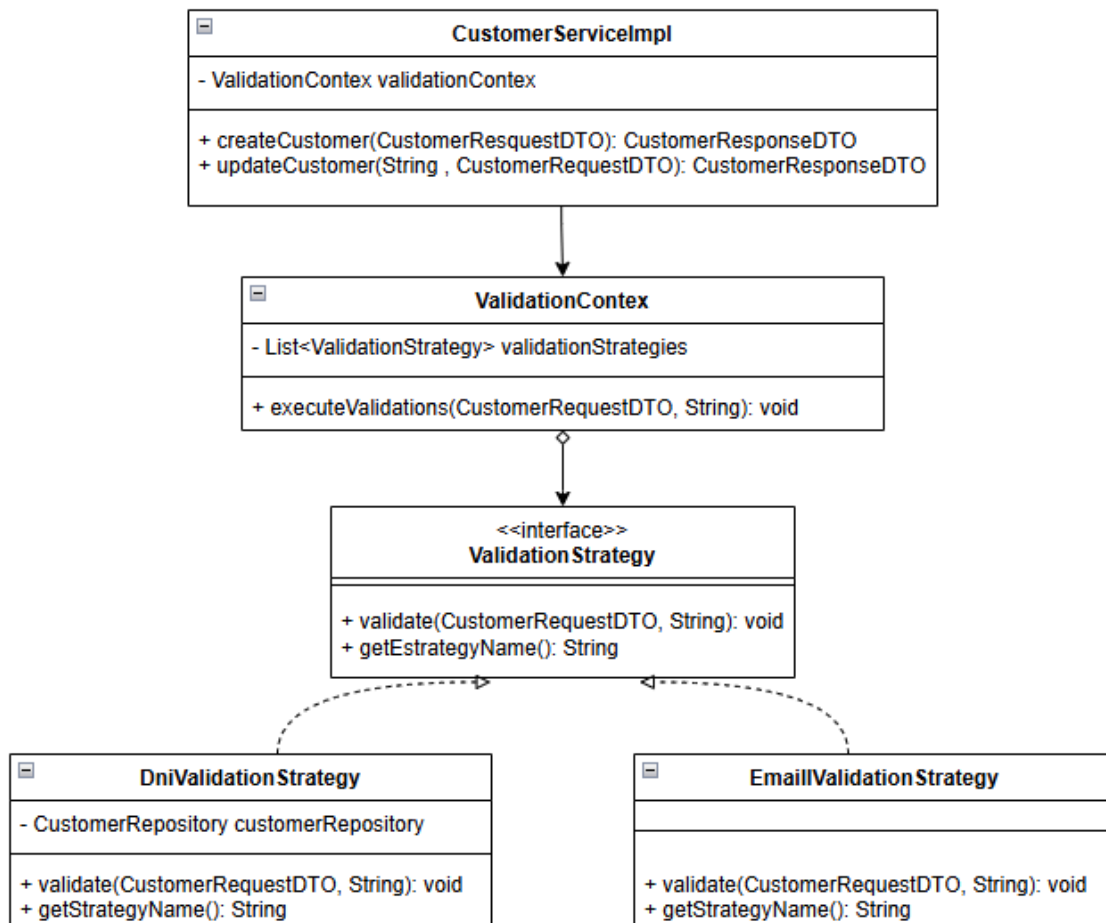
PATRONES DE DISEÑO

PATRÓN STRATEGY

[customer-service]: Este patrón hace que el sistema sea altamente mantenible y extensible, permitiendo agregar nuevas reglas de validación con mínimo impacto en el código existente.

Cumple con el principio Open/Closed, donde se puede agregar nuevas validaciones sin modificar código existente.

- **ValidationStrategy** : la interfaz establece el contrato que todas las estrategias de validación deben de seguir. Con el propósito de definir un contrato común para todas las estrategias de validación, permitiendo que el contexto trabaje con cualquier implementación.
- **DniValitationStrategy** y **EmailValidationStrategy**: son implementaciones concretas de las Estrategias, donde cada estrategia encapsula una regla de validación específica, permitiendo modificar o extender validaciones sin afectar otras partes del sistema.
- **ValidationContext**: Contexto que utiliza las Estrategias, actúa como orquestador, ejecutando todas las estrategias registradas sin conocer los detalles de implementación de cada una.
- **CustomerServiceImpl**: El servicio delega las validaciones al contexto, manteniendo su código limpio y enfocado en la lógica de negocio principal.

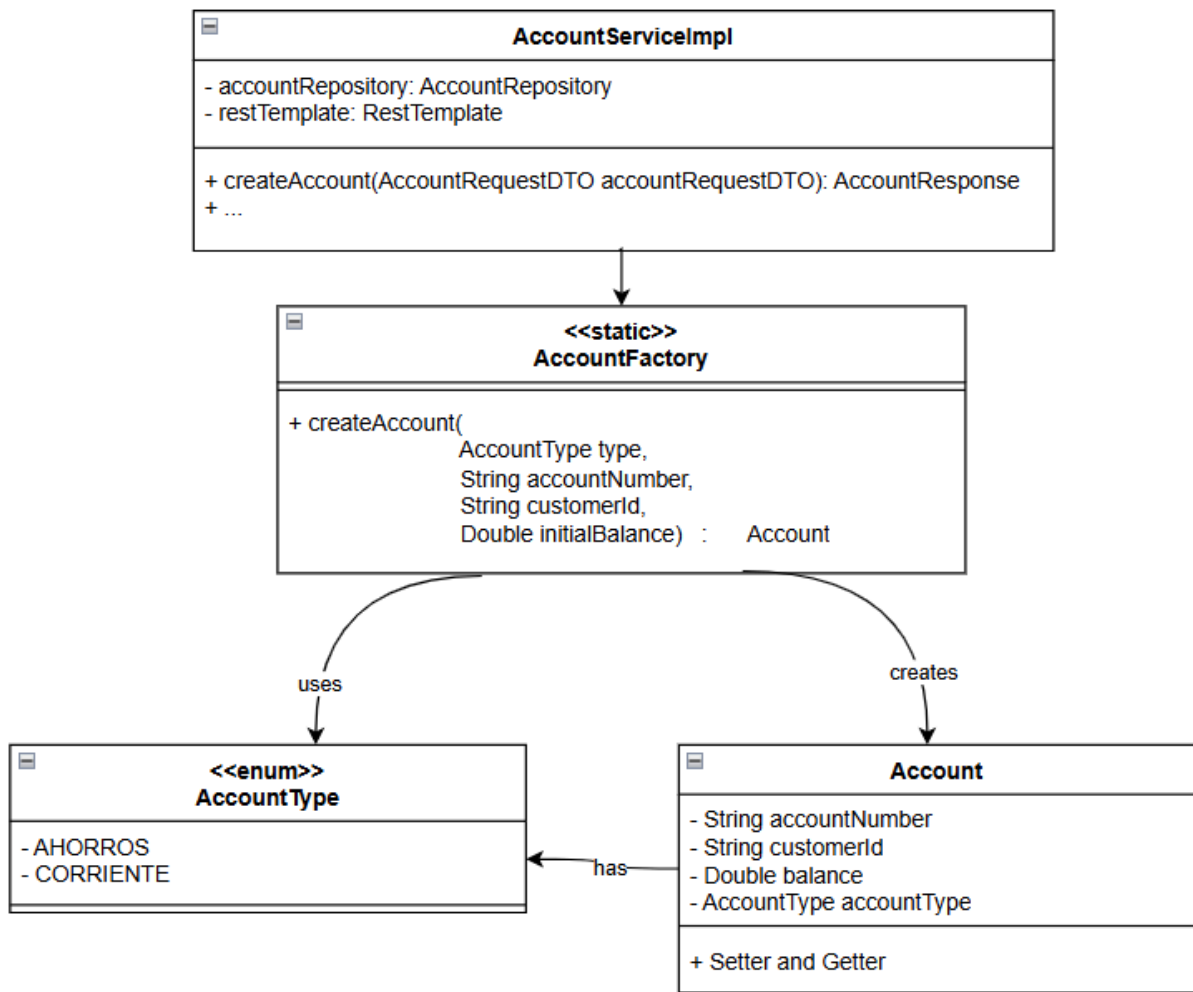


Patrón Factory Method

[account-service]: Se utiliza un método estático que centraliza la creación de cuentas. Esto evita que la lógica de construcción (validación de parámetros, inicialización de valores por defecto) se repita en distintos lugares del código.

Problema que resuelve: evita la duplicación de lógica de creación de objetos y facilita el mantenimiento.

- **AccountServiceImpl**: depende de **AccountFactory** para crear objetos de **Account**.
- **AccountFactory**: utiliza el enum **AccountType** para decidir qué lógica aplicar. Asimismo, crea instancias de **Account** con configuraciones específicas.
- **Account**: tiene relación de composición con **AccountType**.



PATRÓN STRATEGY

[transaction-service]: Este patrón hace que el sistema sea altamente mantenible y extensible, permitiendo agregar nuevas reglas de validación con mínimo impacto en el código existente.

Cumple con el principio Open/Closed, donde se puede agregar nuevas validaciones sin modificar código existente.

- **TransactionStrategy** : la interfaz establece el contrato que todas las estrategias de validación deben de seguir. Con el propósito de definir un contrato común para todas las estrategias de transacción, permitiendo que el contexto trabaje con cualquier implementación.
- **DepositStrategy**, **WithdrawalStrategy** y **TransferStrategy**: son implementaciones concretas de las Estrategias, donde cada estrategia encapsula una regla de transacción específica, permitiendo modificar o extender validaciones sin afectar otras partes del sistema.
- **ValidationContext**: Contexto que utiliza las Estrategias, actúa como orquestador, ejecutando todas las estrategias registradas sin conocer los detalles de implementación de cada una.
- **TransactionServiceImpl**: El servicio delega las validaciones al contexto, manteniendo su código limpio y enfocado en la lógica de negocio principal.

